CSE 489/589
Programming Assignment 1 Report
Text Chat Application

Notes: (IMPORTANT)

- ➔ Your submission will **NOT** be graded without submitting this report. It is required to use this report template.
- ➔ One of your group members select <File> - <Make a copy> to make a copy of this report for your group, and share that Google Doc copy with your teammates so that they can also edit it.
- ➔ Report your work in each section (optional). **Describe** the method you used, the obstacles you met, how you solved them, and the results. You can take screenshots at key points. There are NO hard requirements for your description.
- ➔ For a certain command/event, if you successfully implemented it, **attach the screenshot of the result from the grader (required).** You will get full points if it can pass the corresponding test case of the automated grader.
- ➔ For a certain command/event, if you tried but failed to implement it, properly describe your work. If you can get some points (not full) from the grader, also **attach the screenshot of the result from the grader (required).** We will partially grade it based on the work you did.
- ➔ Do **NOT** claim anything you didn't implement. If you didn't try on a certain command or event, leave that section blank. **We will run your code**, and if it does not match the work you claimed largely, you and your group won't get any partial grade score for this WHOLE assignment.
- ➔ This report has 10 bonus points. Bonus points grading will be based on your **description** in each section, including the sections where you got full points from the grader.
- ➔ Maximum score: 100 + 10 = 110

1. Group and Contributions

- Name of member 1: **Naveen Veeravalli**
  - UBITName: nveerava
  - Contributions: Everything

2. SHELL Functionality

[5.0] Application Startup (startup)

Grader screenshot:

```
embankment {~/pa1-nveerava/pa1} > ./assignment1 client 5555
Only acceptable modes are:
s - to run the application as a server
c - to run the application as a client
embankment {~/pa1-nveerava/pa1} >
```

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 69000
Port should be between 1024 and 65535
stones {~/pa1-nveerava/pa1} >
```

Description:
(Describe how to start your application as a server/client, and how your application accept and process SHELL commands)

The application named `assignment1` can act as both a *client* or a *server* depending on the mandatory argument passed when running the application. In addition to this, the application also requires a port number to bind to when acting as either a server or a client.

`{~/pa1} $ > ./assignment1 <c|s> <port_number>`

If the first argument is any other string than "`c`" or "`s`", the application won't start. If the port number is not a valid port number (i.e below `1024` or above `65535`) the application won't start. Whenever the application does not start, it instead prints out the `usage message` to the `stderr` and exits. The screenshots for the *usage message* is shown below:

3. Test results

[0.0] AUTHOR (author)

Your submission will not be graded if the AUTHOR command fails to work.
Grader screenshot:

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
AUTHOR
[AUTHOR:SUCCESS]
I, pa1-nveerava, have read and understood the course academic integrity policy.
[AUTHOR:END]
```

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
AUTHOR
[AUTHOR:SUCCESS]
I, pa1-nveerava, have read and understood the course academic integrity policy.
[AUTHOR:END]
```

Description:
(Describe your work here…)
The commands AUTHOR, IP and PORT were split into individual functions defined in a separate translation unit. The translation unit is named "`utility_funcs.c`". Since this unit is compiled along with the other translation units for the application, the application running as either a server or as a client can access them and there by saving development time.

[5.0] IP (ip)

Grader screenshot:

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
AUTHOR
[AUTHOR:SUCCESS]
I, pa1-nveerava, have read and understood the course academic integrity policy.
[AUTHOR:END]
IP
[IP:SUCCESS]
IP:128.205.36.46
[IP:END]
```

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
AUTHOR
[AUTHOR:SUCCESS]
I, pa1-nveerava, have read and understood the course academic integrity policy.
[AUTHOR:END]
IP
[IP:SUCCESS]
IP:128.205.36.34
[IP:END]
```

Description:
(Describe your work here…)
As described in the AUTHOR section, the function for IP is also implemented in the translation unit `utility_funcs.c`. The function is named `print_ip()`. The way this function works is by opening up a UDP socket to one of `google.com`'s DNS servers. Once the UDP packet is created, we can extract the IP address of the host by inspecting the socket and retrieving the hostname using the `getsockname()` function. This function fills out a `sockaddr` address

structure for the UDP socket and the IP address can be extracted from this `sockaddr structure` using the `inet_ntop()` function.

[2.5] PORT (port)

Grader screenshot:

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
AUTHOR
[AUTHOR:SUCCESS]
I, pa1-nveerava, have read and understood the course academic integrity policy.
[AUTHOR:END]
IP
[IP:SUCCESS]
IP:128.205.36.46
[IP:END]
PORT
[PORT:SUCCESS]
PORT:5555
[PORT:END]
```

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
AUTHOR
[AUTHOR:SUCCESS]
I, pa1-nveerava, have read and understood the course academic integrity policy.
[AUTHOR:END]
IP
[IP:SUCCESS]
IP:128.205.36.34
[IP:END]
PORT
[PORT:SUCCESS]
PORT:7777
[PORT:END]
```

Description:

(Describe your work here…)

As described in the AUTHOR section, the function for PORT is also implemented in the translation unit `utility_funcs.c`. The function is named `print_port()`. This function accepts the socket's file descriptor as an argument and then retrieves the socket's info using the `getsockname()` function. This function fills out the `sockaddr` address structure for the socket and the PORT number can be extracted from this `sockaddr structure` using the ntohs`()` function on the `sockaddr struct` member `sin_port`.

[20.0] LIST (_list)

Grader screenshot:

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
```

```
highgate {~/pa1-nveerava/pa1} > ./assignment1 c 2222
LIST
```

After Highgate logs in first followed by Euston

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
```

```
highgate {~/pa1-nveerava/pa1} > ./assignment1 c 2222
LIST
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
[LIST:END]
```

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LIST
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
```

Description:
(Describe how you implement the LOGIN and LIST commands)

Before explaining how LOGIN and LIST are implemented, I am going to describe how information is exchanged between the server and clients.

Data is exchanged between the client and servers as a `packet`. This interface is implemented in the file `packet.c` and is exposed through `packet.h` file. A packet object is an opaque data structure. Its interface provides the following functions:

- `make_packet()`: Which creates a packet object.
- `free_packet()`: Which releases the memory associated with a packet object
- `send_packet()`: Which "serializes" a packet object and sends it over a socket described by its parameter.
- `receive_packets()`: Which receives packet objects (in serialized format) from a socket into a circular (described next) data structure.
- `get_length()`: Returns the length of the serialized packet object.
- `get_command()`: Returns the "command" representing the packet object.
- `get_message()`: Returns the data corresponding to the command for either the client or the server to process.

A serialized packet has the following structure:
The (up to) first 4 bytes represent the size of the message (as returned by the function `get_message()`) in serialized format. Next 16 bytes are used for the command. This part is always fixed at 16 bytes (any left over bytes are zero'ed out). After that is the message data (which is variable in length and null terminated).
For example, the serialized version of a packet representing a SEND to 172.206.36.35 with the message "HOW ARE YOU?" would look like:

2 7 S E N D \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 1 7 2 . 2 0 6 . 3 6 . 3 5 \30

H O W A R E Y O U ? \0

Here, <RS> represents the ASCII character 30 (0x1E) signifying a record separator.

Additionally, the project also uses a circular data structure to buffer packets received by either the client or a socket. It is defined in the translation unit `circular_buffer.c` and its interface is provided in `circular_buffer.h` Each buffer is large enough to simultaneously hold **two** `packet` objects (serialized) of maximum length (The maximum length is defined as a macro constant). This buffer is a *circular data structure* and is also an opaque object. The interface provides the following functions:

- `create_buffer()`: Creates and initializes a circular buffer object.
- `free_buffer()`: Releases the memory associated with a circular buffer.
- `get_max_packet_size()`: Returns the length of the maximum size of a packet that the buffer accepts.
- `buffer_push()`: Pushes the data of a packet (serialized) onto a circular buffer object.
- `buffer_pop_packet()`: Pops a serialized packet from the circular buffer object.

The main rationale behind using the `packet` and `circular_buffer` interface is explained in the book “Beej’s Guide to Network Programming” in Chapter 7 sections 7.5 and 7.6.

Now let’s get back to understanding how LOGIN and LIST commands are implemented:

The basic idea is that the client `connect()`s to the server with the server’s IP and Port number and the server `listen()`s to new connections and `accept()`s any new requests. After the server `accept()`s the connection, it creates a new `struct user` (defined in `global.h`) and fills in the structure with the new connection’s socket file descriptor, the connection’s IP address, Port number and an enumerator indicating whether the connection is `ONLINE` or `OFFLINE`. Once this new struct is successfully filled with client’s info, we store a reference to this structure in an array of `struct user*` named `users`. Once this is done, the server serializes the currently logged in clients into a string using the function `generate_users_string()` defined in the translation unit `serialize.c`. This function creates a string of the following format:

“COUNT<US>USER1->IP<RS>USER1->HOSTNAME<RS>USER1->PORT<US>USER2->IP<RS>USER2->HOSTNAME<RS>...”

As you can see, each user’s information is separated by a unit separator character (ASCII 31) and the various user fields (IP, Hostname and Port Number) themselves are separated by a record separator character (ASCII 30). Once this string is generated, the server creates a `packet` with LOGIN command and the above string and `send_packet()`s it to the client which then `receive_packet()`s it and deserializes it using the `parse_users_string()` function (also defined in the file `serialize.c`) that extracts the count of users and then the

client data sequentially and stores this information in its own copy of array of `struct user*` also called `users`.

If the client is LOGging in after a logout, the server also checks to see if the `user` has any backlog messages (stored in `user→backlog_list`) and generates a serialized string of all their backlog messages using the function `generate_backlog_string()` (defined in `serialize.c`). After that it sends a BACKLOG `packet` to the client (this is in addition to the LOGIN packet above). The backlog string in the packet is of the following format:

COUNT<US>USER1→IP<RS>USER1→MSG<US>USER2→IP<RS>USER2→MSG<US>...

As you can see, the message starts with the count of backlog messages. Each unit is separated using the ASCII character 31 (Unit separator) while the data for each message (ie, the sender's IP address and the message) are separated by the ASCII character 30 (Record separator)

On the client's side, upon receiving the BACKLOG `packet`, it parses the message in the packet using the function `parse_backlog_string()` (defined in `serialize.c`) and proceeds to display them to the user in the manner described in the template.

The LIST command for both the client and server uses the function `list_users()` to display the currently logged in clients. This function is defined in the file `user_ops.c`. Since any new client is stored at any empty index in users array when they established a new connection, it is possible that not all users are listed consecutively in the array (i.e, some of the array indexes might not point to any valid `user` object), thus the list needs to be sorted before the users can be displayed in ascending order. This is done using the standard C library function `qsort()`and pass a comparison function `compare_users()` (defined in `utility_funcs.c`) which compares two user objects based on their port numbers.

[2.0] LOGIN Exception Handling (exception_login)

Grader screenshot:

```
underground {~/pa1-nveerava/pa1} > ./assignment1 c 3333
LOGIN stones.cse.buffalo.edu 5555five
[LOGIN: ERROR]
[LOGIN: END]
```

```
underground {~/pa1-nveerava/pa1} > ./assignment1 c 3333
LOGIN stones.cse.buffalo.edu 5555five
[LOGIN: ERROR]
[LOGIN: END]
LOGINN 128.205.36.46 5555
134
[LOGIN: ERROR]
[LOGIN: END]
```

Description:
(Describe your work here…)
To check for the exceptions for LOGIN command, the input string is tokenized using the standard library function `strtok()`. The input string as provided by the user is tokenized using the space character (“ “) as the delimiter.

On the client’s side, after checking that the first token is indeed exactly “LOGIN”, the second token is extracted and it’s validity as an IP address is confirmed by passing it to the function `inet_pton()` and checking whether it can be converted to ***Network Byte order*** or not.

Similarly, the next token is extracted and checked to see if it is a number using `strtol()` function (This function converts a string to a long integer and stores the first byte representing the non-digit character into its third argument `endptr`). It is an exception (and hence an error if the third token is a number along with strings like “3498str” and `strtol()` helps in finding out such strings).

[5.0] REFRESH (refresh)

Grader screenshot:

```
underground {~/pa1-nveerava/pa1} > ./assignment1 c 3333
LOGIN stones.cse.buffalo.edu 5555five
[LOGIN: ERROR]
[LOGIN: END]
LOGINN 128.205.36.46 5555
134
[LOGIN: ERROR]
[LOGIN: END]
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
```

This is the output from Highgate after running REFRESH and LIST commands:

```
highgate {~/pa1-nveerava/pa1} > ./assignment1 c 2222
LIST
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
[LIST:END]
REFRESH
[REFRESH:SUCCESS]
[REFRESH:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
```

Description:
(Describe your work here…)
On the server's side, when it receives a REFRESH `packet` from the client, just like in the case of a new connection, the server creates a serialized format of the users list using `generate_users_string()` function and `send_packet()`s it to the client.

Over on the client's side, upon `receive_packet()` of the REFRESH `packet`, the client calls the `parse_user_data()` to deserialize the string and creates a `struct user` and stores their references in the `users` array. Before doing this however, the client resets the entire `users` array by initializing them to `NULL`.

[2.5] EXIT (exit)

Grader screenshot:
Euston EXITs now:

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LIST
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
EXIT
[EXIT:SUCCESS]
[EXIT:END]
euston {~/pa1-nveerava/pa1} >
```

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
```

Description:
(Describe your work here…)
When a client receives an EXIT command from the user who invoked it, it `send_packet()`s an EXIT `packet` to the server. Additionally, it frees all the memory allocated on the heap during its runtime (such as the *circular buffer*, list of Ips it blocked, the `users` array, etc.) and then proceeds to `close()` the connection.

The server upon `receive_packet()` the EXIT packet and parsing it, proceeds to `free()`s the resources it allocated for the particular user (including the *circular buffer* `user→cb`, backlog messages if any, list of blocked IP addresses and finally the `user` object itself).

After this, the server `close()`s the socket file descriptor associated with that connection and removes it from the `master_list` for the `select()` call.

[15.0] SEND (send)

Grader screenshot:
SENDing a message from Underground to Highgate:

```
underground {~/pa1-nveerava/pa1} > ./assignment1 c 3333
LOGIN stones.cse.buffalo.edu 5555five
[LOGIN: ERROR]
[LOGIN: END]
LOGINN 128.205.36.46 5555
134
[LOGIN: ERROR]
[LOGIN: END]
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
SEND 128.205.36.33 Hello Highgate. This is Underground!
[SEND:SUCCESS]
[SEND:END]
```

Description:
(Describe your work here…)
When a client receives a SEND command as input, it first checks if the client is logged into the server or not. If it is indeed logged in, then the application proceeds to parse the input string using the `strtok()` function. The first token is checked to see if it matches exactly with the "LOGIN" string. If the first token passes the above check, the second token is extracted. The second token is checked for the following cases:

- **String represents valid IP address**: This is done by passing the string to the `inet_pton()` function and verify if the provided IP address can be successfully converted to Network Byte order format or not.
- **The IP address does not represent the Client's own IP address**: We compare the IP address in the input against the client's own IP address which is stored in a variable named `host_ip`. It is an ERROR if they are a match.

Once the second string is processed, the third token from the input is processed according to the following conditions:

- **The string is no longer than 256 characters**.
- **The string contains only valid ASCII characters**. This is done using the function `is_valid_ascii()`.

Now that the input is processed and checked to follow that it is indeed in the expected format, the client creates a SEND `packet` and `send_packet()` it to the server. The packet's message which is created using the function `generate_msg_string()` (defined in `serialize.c`) has the following format:

RECEIVER_IP<RS>MSG_TO_SEND\0

where <RS> represents the ASCII character 0x1E (record separator).

On the server's side, when it `receive_packets()` a SEND `packet` from a client, it parses the packet's message using the function `parse_msg_string()` (defined in `serialize.c`) to extract the destination IP address and the message. After extracting the destination IP address, it checks if the recipient has blocked the sender or not. If the receiver has indeed blocked the sender, the server simply updates the statistics for the sender and not the receiver and discards the packet. Next it checks if the destination client is `ONLINE` and if it is, proceeds to create a SEND `packet` with the following format (once again the message is generated using generate_msg_string()):

SENDER_IP<RS>MSG_RECEIVED\0

The server then `send_packet()` it to the recipient and proceeds to update the statistics for this receiver (ie, `user→msg_recv`) and also the sender (ie `user→msg_sent`). If the recipient is `OFFLINE`, the server, simply adds the message to the `user`'s backlog list (`user→backlog_msg_list`) and updates the statistics for the sender and receiver. Finally, it displays an EVENT on its output indicating the sender's IP, receiver's IP and the message being relayed in the format provided in the template.

On the recipient's side, when they receive a SEND `packet`, the application simply parses it and displays the message and the sender's IP as an EVENT in the format provided in the template

[EVENT]: Message Relayed

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
```

Description:

(Describe your work here…)

As described in the SEND section, the server displays this EVENT on the standard output whenever it successfully sends a `packet` to the receiver. In this case, the EVENT is displayed in the format described in the report template.

Additionally, this event is also displayed whenever the server BROADCASTs a message successfully to each client (except for the client who BROADCASTed the message). In this case, the EVENT is displayed as described in the template where the sender's address is shown as `255.255.255.255`.

[EVENT]: Message Received

```
highgate {~/pa1-nveerava/pa1} > ./assignment1 c 2222
LIST
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
[LIST:END]
REFRESH
[REFRESH:SUCCESS]
[REFRESH:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
[RECEIVED:SUCCESS]
msg from:128.205.36.36
[msg]:Hello Highgate. This is Underground!
[RECEIVED:END]
```

Description:
(Describe your work here…)
As described in the SEND section, a recipient client displays this EVENT on the standard output after successfully parsing a SEND `packet`. Whether the sender sent a simple SEND message or a BROADCAST message, on the recipient's side, it will always display the EVENT in the same way. That is to say, the recipient would not know if the sender SEND or BROADCAST the message.

[10.0] BROADCAST (broadcast)

Grader screenshot:

```
embankment {~/pa1-nveerava/pa1} > ./assignment1 c 9999
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BROADCAST Hello everybody. This is Embankment!
[BROADCAST:SUCCESS]
[BROADCAST:END]
```

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
[RELAYED:SUCCESS]
msg from:128.205.36.35, to:255.255.255.255
[msg]:Hello everybody. This is Embankment!
[RELAYED:END]
```

```
highgate {~/pa1-nveerava/pa1} > ./assignment1 c 2222
LIST
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
[LIST:END]
REFRESH
[REFRESH:SUCCESS]
[REFRESH:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
[RECEIVED:SUCCESS]
msg from:128.205.36.36
[msg]:Hello Highgate. This is Underground!
[RECEIVED:END]
[RECEIVED:SUCCESS]
msg from:128.205.36.35
[msg]:Hello everybody. This is Embankment!
[RECEIVED:END]
```

```
underground {~/pa1-nveerava/pa1} > ./assignment1 c 3333
LOGIN stones.cse.buffalo.edu 5555five
[LOGIN: ERROR]
[LOGIN: END]
LOGINN 128.205.36.46 5555
134
[LOGIN: ERROR]
[LOGIN: END]
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
SEND 128.205.36.33 Hello Highgate. This is Underground!
[SEND:SUCCESS]
[SEND:END]
[RECEIVED:SUCCESS]
msg from:128.205.36.35
[msg]:Hello everybody. This is Embankment!
[RECEIVED:END]
```

Description:
(Describe your work here…)
When a client receives a BROADCAST input from its user, it tokenizes the input as described in the previous sections using `strtok()`. This time however, there would be no recipient IP address in the input. The message itself is checked for the same conditions described in the SEND section (ie, the message should be no longer than 256 characters and should only contain valid ASCII characters).

The server upon receiving a BROADCAST `packet`, performs the exact same sequence of steps described in the SEND section with the *only difference* being that it does it for **all the clients in its `users` array excluding the `user` who BROADCASTed the message**.

Upon receiving the SEND `packets`, each recipient parses their respective `packets` and displays them in the format described in the template.

The same set EVENTs are respectively displayed on both the server (as many EVENTs as there are `ONLINE` users) and on the recipients' side.

[5.0] STATISTICS (statistics)

Grader screenshot:

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
[RELAYED:SUCCESS]
msg from:128.205.36.35, to:255.255.255.255
[msg]:Hello everybody. This is Embankment!
[RELAYED:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          0       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
```

Description:

(Describe how you implement the LOGIN and LIST commands)

Upon receiving the STATISTICS command from the input, the server proceeds to verify that the input is exactly “STATISTICS” and nothing more or less. After validating the input, it then proceeds to sort the list of `users` in its `users` array according to their port numbers. This is once again achieved using the standard C library function `qsort()` and passing the auxiliary function `compare_users()` (defined in `utility_funcs.c`) which allows two user objects to be compared w.r.t their port numbers. Once that is done, it displays the required statistics (ie, messages sent: `user→msg_sent`, messages received: `user→msg_recv` and login status: `user→status`) for each client in its `users` list sorted in ascending order of their port numbers in the format provided in the template.

[5.0] BLOCK (block)

Grader screenshot:

Euston BLOCKs Highgate after which Highgate SENDs a message to Euston.

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BLOCK 128.205.36.33
[BLOCK:SUCCESS]
[BLOCK:END]
```

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu           128.205.36.33       2222
2    euston.cse.buffalo.edu             128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu           128.205.36.33       2222
2    underground.cse.buffalo.edu        128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
[RELAYED:SUCCESS]
msg from:128.205.36.35, to:255.255.255.255
[msg]:Hello everybody. This is Embankment!
[RELAYED:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu           0       2       logged-in
2    underground.cse.buffalo.edu        1       1       logged-in
3    embankment.cse.buffalo.edu         2       0       logged-in
[STATISTICS:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu           1       2       logged-in
2    underground.cse.buffalo.edu        1       1       logged-in
3    euston.cse.buffalo.edu             0       0       logged-in
4    embankment.cse.buffalo.edu         2       0       logged-in
[STATISTICS:END]
```

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BLOCK 128.205.36.33
[BLOCK:SUCCESS]
[BLOCK:END]
```

Description:
(Describe your work here…)
When a client receives a BLOCK command from the input, it parses the input using `strtok()` function. After the tokenization, it compares the first token to exactly match the string “BLOCK”. The second token is then validated to be an IP address using the `inet_pton()` function. After that it checks if the IP address provided in the input exists in the list of `users` stored by the application. If the IP address is not present in the `users` list, it is an ERROR. If the IP address is present in the `users` list, it then sends a BLOCK `packet` to the server with the message of the `packet` being the IP address the client wants blocked. If the IP address is already BLOCKed by the client, no `packet` is sent to the server and the command is simply SUCCESSfully processed.

In order to be fast and responsive, the client maintains an array of `blocked_ips`. When a BLOCK command is successfully processed and the `packet` is sent to the server, it also adds the IP address from the input to this list.

On the server’s side, upon receiving a BLOCK `packet`, and the message from the `packet` is processed. This involves checking if the message contains more tokens than a single IP address and if the IP address is a valid one (using `inet_pton()` function). After that the server adds the IP to the `user`’s blocked IPs list (`user→blocked_ips`).

[5.0] BLOCKED (blocked)

Grader screenshot:
Euston BLOCKs Embankment:

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BLOCK 128.205.36.33
[BLOCK:SUCCESS]
[BLOCK:END]
BLOCK 128.205.36.35
[BLOCK:SUCCESS]
[BLOCK:END]
```

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
[RELAYED:SUCCESS]
msg from:128.205.36.35, to:255.255.255.255
[msg]:Hello everybody. This is Embankment!
[RELAYED:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          0       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          1       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    euston.cse.buffalo.edu            0       0       logged-in
4    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
BLOCKED 128.205.36.34
[BLOCKED:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    embankment.cse.buffalo.edu        128.205.36.35       9999
[BLOCKED:END]
```

Description:

(Describe your work here…)

When the server receives a BLOCKED command from the input, it tokenizes the input using strtok() to ensure that the tokens are in the exact same numbers and in the expected format.

This means that the first token is checked to see if it matches the string “BLOCKED” exactly and then it validates the IP address using inet_pton().

After this validation step, it finds the `user` object representing this IP address and then iterates over the `user`’s blocked list of IP addresses (`user→blocked_list`). Additionally, it creates a list of `users` representing these IP addresses and proceeds to make copies of the `user` objects

related to these IP addresses and fills them into the list. After that this list of blocked users is sorted w.r.t their port numbers and prints them to the output in the desired format.

[2.5] UNBLOCK (unblock)

Grader screenshot:

Euston UNBLOCKs Embankment but has Highgate still BLOCKed

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BLOCK 128.205.36.33
[BLOCK:SUCCESS]
[BLOCK:END]
BLOCK 128.205.36.35
[BLOCK:SUCCESS]
[BLOCK:END]
UNBLOCK 128.205.36.35
[UNBLOCK:SUCCESS]
[UNBLOCK:END]
```

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
[RELAYED:SUCCESS]
msg from:128.205.36.35, to:255.255.255.255
[msg]:Hello everybody. This is Embankment!
[RELAYED:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          0       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          1       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    euston.cse.buffalo.edu            0       0       logged-in
4    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
BLOCKED 128.205.36.34
[BLOCKED:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    embankment.cse.buffalo.edu        128.205.36.35       9999
[BLOCKED:END]
BLOCKED 128.205.36.34
[BLOCKED:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
[BLOCKED:END]
```

Description:
(Describe your work here…)
The UNBLOCK command is processed in a similar fashion to the BLOCK command except that it does the opposite. Also, instead of sending a BLOCK `packet` to the server, it sends an UNBLOCK `packet` to the server. Finally, it sends an UNBLOCK `packet` and then removes the IP address from the `blocked_ips` local array so that it can immediately start processing requests pertaining to the recently UNBLOCKed IP from the input.

Similarly, on the server's side, after validating the message in the UNBLOCK `packet`, the IP address is removed from the user's blocked IPs list (`user->blocked_ips`).

[2.5] LOGOUT (logout)

Grader screenshot:
Euston LOGOUT:

```
euston {~/pa1-nveerava/pa1} > ./assignment1 c 7777
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BLOCK 128.205.36.33
[BLOCK:SUCCESS]
[BLOCK:END]
BLOCK 128.205.36.35
[BLOCK:SUCCESS]
[BLOCK:END]
UNBLOCK 128.205.36.35
[UNBLOCK:SUCCESS]
[UNBLOCK:END]
LOGOUT
[LOGOUT:SUCCESS]
[LOGOUT:END]
```

Notice both LIST and STATISTICS command on the server:

```
stones {~/pa1-nveerava/pa1} > ./assignment1 s 5555
LIST
[LIST:ERROR]
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    euston.cse.buffalo.edu            128.205.36.34       7777
[LIST:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
[LIST:END]
[RELAYED:SUCCESS]
msg from:128.205.36.36, to:128.205.36.33
[msg]:Hello Highgate. This is Underground!
[RELAYED:END]
[RELAYED:SUCCESS]
msg from:128.205.36.35, to:255.255.255.255
[msg]:Hello everybody. This is Embankment!
[RELAYED:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          0       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          1       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    euston.cse.buffalo.edu            0       0       logged-in
4    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
BLOCKED 128.205.36.34
[BLOCKED:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    embankment.cse.buffalo.edu        128.205.36.35       9999
[BLOCKED:END]
BLOCKED 128.205.36.34
[BLOCKED:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
[BLOCKED:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
STATISTICS
[STATISTICS:SUCCESS]
1    highgate.cse.buffalo.edu          1       2       logged-in
2    underground.cse.buffalo.edu       1       1       logged-in
3    euston.cse.buffalo.edu            0       0       logged-out
4    embankment.cse.buffalo.edu        2       0       logged-in
[STATISTICS:END]
```

Description:
(Describe your work here…)
When a client application receives a LOGOUT command from the input, it is first processed by tokenizing the input and comparing the *first and only* token with the string “TOKEN”. After that it creates a LOGOUT `packet` and `send_packet()` it to the server.

Once the `packet` is sent out, the client sets the state variable `logged_in` to `FALSE` so that any user command involving communicating over the socket with the server don’t work.

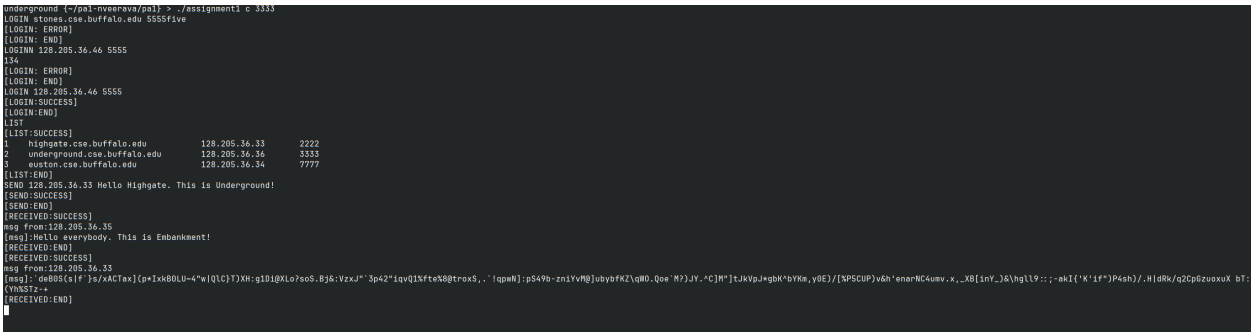
On the server’s side, upon receiving the LOGOUT `packet`, the server checks to see if the `user` has any backlog messages (which it shouldn’t) after which it simply sets the `user`’s logged_in field to `OFFLINE`. After that any message this user receives is added to its backlog list (`user->backlog_list`) until the user logs back in again.

[5.0] (buffer)

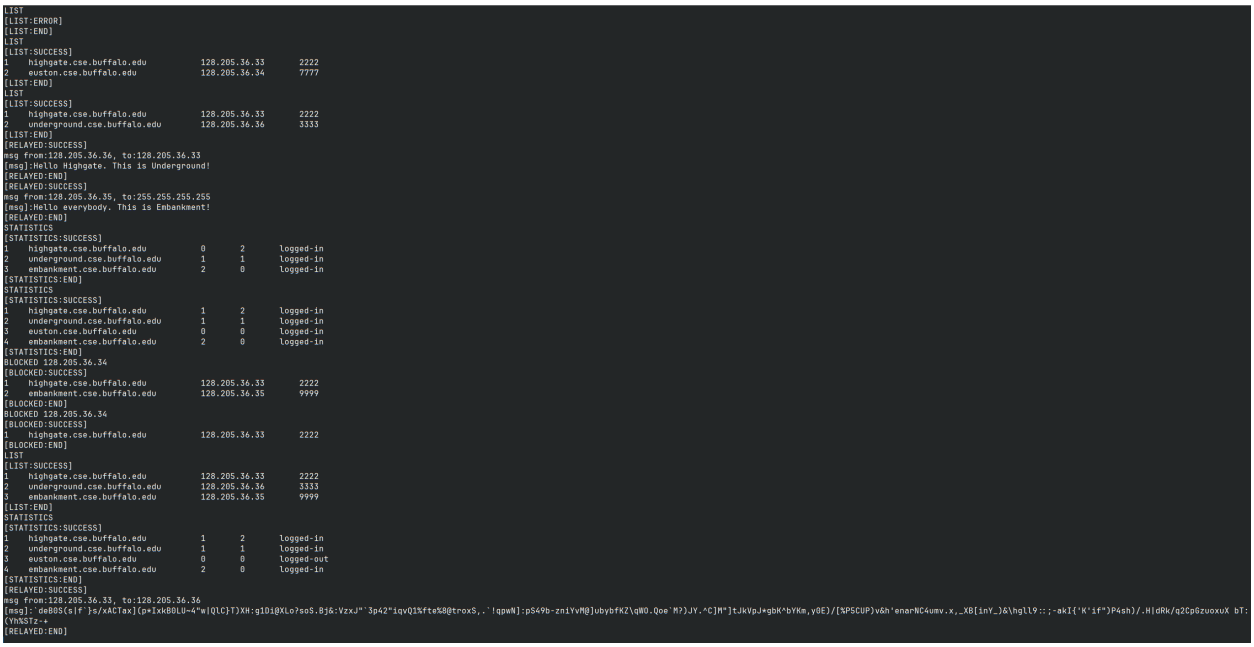
Grader screenshot:
This is the message to send (exactly 256 bytes long):
“**`deB0S(s|f`}s/xACTax](p*IxkB0LU~4"w|QlC}T)XH:g1Di@XLo?soS.Bj&:VzxJ"`3p42"iqvQ1%fte%8@troxS,.`!qpwN]:pS49b-zniYvM@]ubybfKZ\qWO.Qoe`M?)JY.^C]M"]tJkVpJ*gbK^bYKm,y0E)/[%P5CUP)v&h'enarNC4umv.x,_XB[inY_)&\hgll9::;-akI{'K'if")P4sh)/.H|dRk/q2CpGzuoxuX bT:(Yh%Stz-+**”

This message is SENT from Highgate to Underground.





This is a screenshot of the server:



Description:

(Describe how your server stores/buffers messages)

The primary reason for implementing and using the `circular_buffer` data structure is to help with the problem of buffering. Fundamentally, this application is implemented over TCP sockets. TCP sockets usually break the data that needs to be sent over the network into multiple chunks if the original data size is larger than its buffer sizes (this is automatically maintained by the kernel and applications in the user space don't get to control it. The user space application can suggest the kernel to use a specific buffer size but the kernel is free to ignore it). When sending large messages through sockets (for example in this project's case, a message of size 256 bytes), the actual packet might be broken over multiple TCP packets.

Using a circular_buffer structure solves the problem of receiving partial data entirely as the buffer is large enough to hold two theorectical maximum `packets`. While at the same time, the interface for `packet` provides the application with functions to receive complete `packets`

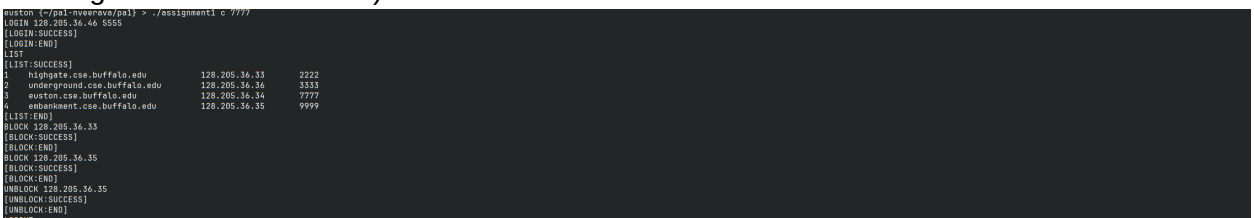
(`receive_packets()`). The `packet` interface uses this `circular_buffer` structure to buffer potentially incomplete data received over a TCP socket while only allowing an application to extract data once an entire packet is available from the `circular_buffer`. This means that as pointed above, the `circular_buffer` only ever needs to be 2 maximum length `packets` long.

This concept is explained in **Chapter 7** sections **7.5** and **7.6** on *data serialization* and *encapsulation* in the book *Beej's Guide to Network Programming*. As described in the section on LOGIN and LIST earlier, each `packet`'s serialized version starts with the length of the application message in text format. For example if the user's data is 24 bytes long, the first two bytes of a serialized `packet` will be "24". This logic allows the circular_buffer's function `buffer_pop_packet()`. to first check for the size of the message before determining whether a complete `packet` has been received or not. As you can see in the screenshot below, a client can SEND or BROADCAST the theoretical maximum size messages (256 bytes) without any problem.

[2.0] SEND Exception Handling (exception_send)

Grader screenshot:

Please check the screenshots for BLOCK which showcase the validations Server performs before RELAYing a message to the recipient. Below is the screenshot when BROADCASTING a message that is more than 256 bytes long. Same message from the previous section with one extra character (so the message is 257 bytes long. Just appended an 'a' to the end of the message from buffer section).



Description:
(Describe your work here…)

1. **Client-Side Validations and Exceptions**:

- *Message Length and ASCII Validation*: Checks that the message does not exceed 256 characters and contains valid ASCII characters.

- *Recipient IP Validation*: Validates the recipient's IP address for correctness and ensures it's not the client's own IP

2. **Server-Side Validations and Exceptions**:

- *Recipient Availability and Block Status*: Ensures the recipient is available and has not blocked the sender. Messages to unavailable or blocking recipients are handled according to requirements provided in the template while also updating their respective statistics.

[2.0] BLOCK Exception Handling (exception_block)

Grader screenshot:

Description:
(Describe your work here…)

1. **Client-Side Validations and Exceptions**:

- *IP Address Validation*: Ensures the IP address provided for blocking is a valid IPv4 address using `inet_pton()`. This prevents attempts to BLOCK invalid IP addresses.
- *Duplication Check*: Verifies the IP address is not already blocked and exists in the local `blocked_ips` list. This prevents redundant BLOCK requests and ensures only known `users` can be blocked.

2. **Server-Side Validations and Exceptions**:

- *User Location Identification*: Locates the requesting user in the server's `users` list to associate the BLOCK request. Failure to find the `user` results in an ERROR, ensuring that the BLOCK request is associated with a connected `user`.
- *Block List Update*: Attempts to add the specified IP to the requesting `user`'s block list while ensuring that there are no duplicates in the list.

[2.0] BLOCKED Exception Handling (exception_blocked)

Grader screenshot:

```
embankment {~/pa1-nveerava/pa1} > ./assignment1 c 9999
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BROADCAST Hello everybody. This is Embankment!
[BROADCAST:SUCCESS]
[BROADCAST:END]
BLOCK 128.205.36.35
[BLOCK:ERROR]
[BLOCK:END]
```

```
embankment {~/pa1-nveerava/pa1} > ./assignment1 c 9999
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BROADCAST Hello everybody. This is Embankment!
[BROADCAST:SUCCESS]
[BROADCAST:END]
BLOCK 128.205.36.35
[BLOCK:ERROR]
[BLOCK:END]
REFRESH
[REFRESH:SUCCESS]
[REFRESH:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
UNBLOCK 128.205.36.35
[UNBLOCK:ERROR]
[UNBLOCK:END]
UNBLOCK 128.205.36.39
[UNBLOCK:ERROR]
[UNBLOCK:END]
BLOCK 128.205.36.39
[BLOCK:ERROR]
[BLOCK:END]
```

Description:
(Describe your work here…)

- **Invalid IP address:** Ensure that the IP address received from the input is valid (using `inet_pton()`) and also that they are not BLOCKing themselves.
- **Valid but non-existent IP address**: Once the validity of the IP address is confirmed, the server checks that the IP address matches one of `user`'s IP address from its `users` list.

[2.0] UNBLOCK Exception Handling (exception_unblock)

Grader screenshot:

```
embankment {~/pa1-nveerava/pa1} > ./assignment1 c 9999
LOGIN 128.205.36.46 5555
[LOGIN:SUCCESS]
[LOGIN:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
BROADCAST Hello everybody. This is Embankment!
[BROADCAST:SUCCESS]
[BROADCAST:END]
BLOCK 128.205.36.35
[BLOCK:ERROR]
[BLOCK:END]
REFRESH
[REFRESH:SUCCESS]
[REFRESH:END]
LIST
[LIST:SUCCESS]
1    highgate.cse.buffalo.edu          128.205.36.33       2222
2    underground.cse.buffalo.edu       128.205.36.36       3333
3    euston.cse.buffalo.edu            128.205.36.34       7777
4    embankment.cse.buffalo.edu        128.205.36.35       9999
[LIST:END]
UNBLOCK 128.205.36.35
[UNBLOCK:ERROR]
[UNBLOCK:END]
UNBLOCK 128.205.36.39
[UNBLOCK:ERROR]
[UNBLOCK:END]
```

Description:

(Describe your work here…)

1. **Client-Side Validations and Exceptions**:

- *IP Address Validation*: Ensures the IP address provided for blocking is a valid IPv4 address using `inet_pton()`. This prevents attempts to unblock invalid IP addresses. Also ensures that the client is not UNBLOCKing themselves.

2. **Server-Side Validations and Exceptions**:

- *User Location Identification*: Locates the requesting user in the server's `users` list to associate the block request. Failure to find the `user` results in an ERROR, ensuring that the UNBLOCK request is associated with a connected `user`.
- *Block List Update*: Attempts to remove the specified IP to the requesting `user`'s blocked IPs list.